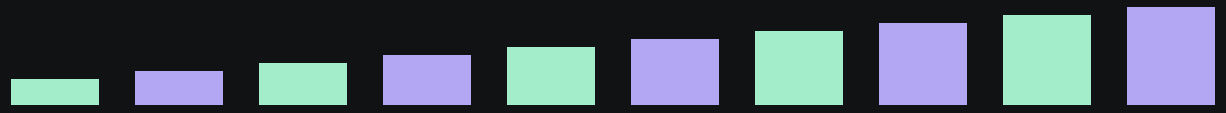
KubeSentinel AI

Observe clearly. Recover with evidence.

Evidence.
Policy.
Verified recovery.



Kubernetes operations, explainable detection and guarded recovery

Engineering portfolio | Release preparation | November 2026

The operational problem

Signals arrive separately; recovery needs shared context

01 Metrics, logs and Kubernetes events describe different parts of the same failure.

02 Unbounded automation can amplify an outage or cross an authorization boundary.

03 Objective: connect evidence, propose bounded actions and verify the outcome.

Engineering differentiators

Authority stays deterministic

01 Separate detection, authorization and execution boundaries

02 Default dry-run, exact-plan approvals, cooldowns and action budgets

03 Durable action evidence and verification before success

04 Explicit provenance: real cluster, synthetic fixture or planning assumption

Technology choices

Independent privilege boundaries; shared typed contracts

01 Experience | Next.js, TypeScript, Tailwind, TanStack Query

02 Application | FastAPI, Pydantic, SQLAlchemy, Alembic

03 State | PostgreSQL durable records; Redis transient coordination

04 Execution | Go controller, versioned Kubernetes CRDs, Helm

05 Operations | Prometheus, Loki, OTel, Grafana, Terraform, Argo CD

High-level architecture

Telemetry informs decisions; policy authorizes execution

01
Cluster signals

02
Ingest + normalize

03
Detect + correlate

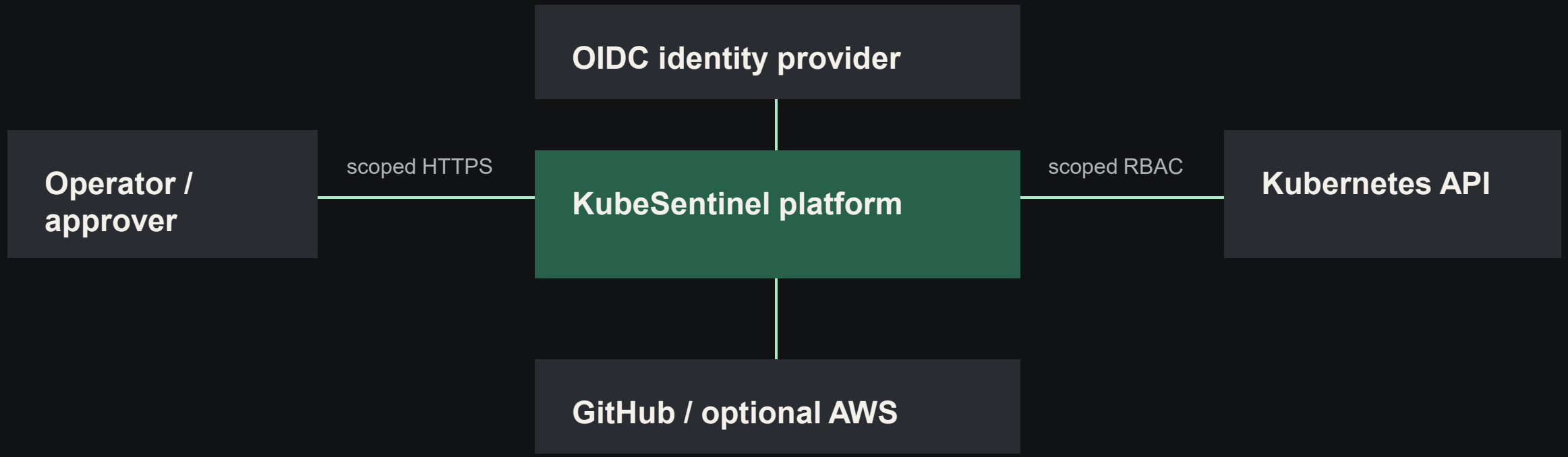
04
Policy + approval

05
Operator + verify

PostgreSQL stores incident and audit state.

C4 context and trust boundaries

People and external systems



User identity, execution and delivery are separate trust boundaries.

C4 containers and API components

A container is a process or store, not a directory promise

Container view



API component view



Execution is independently authorized; stores are not instruction channels.

AWS EKS infrastructure

Validated source; no live deployment

01
Encrypted remote state

02
VPC + private subnets

03
EKS managed nodes

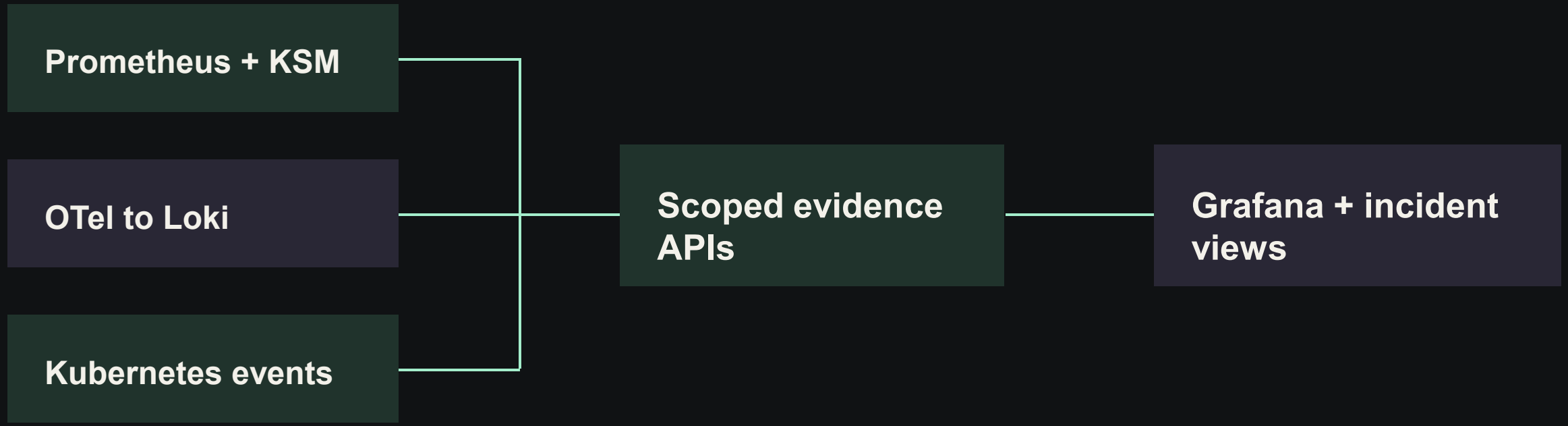
04
Scoped IAM + Pod Identity

05
ECR + CloudWatch

Dev and demonstration roots expose configurable CIDRs, node bounds and access entries.

Observability and incident ingestion

Metrics, logs and events retain distinct semantics



Bounded queries, retention and cardinality; explicit behavior during outages.

Reproducible detection pipeline

Temporal splits and provenance precede model complexity

01
Timestamp + provenance

02
Window + rolling features

03
Time-based split

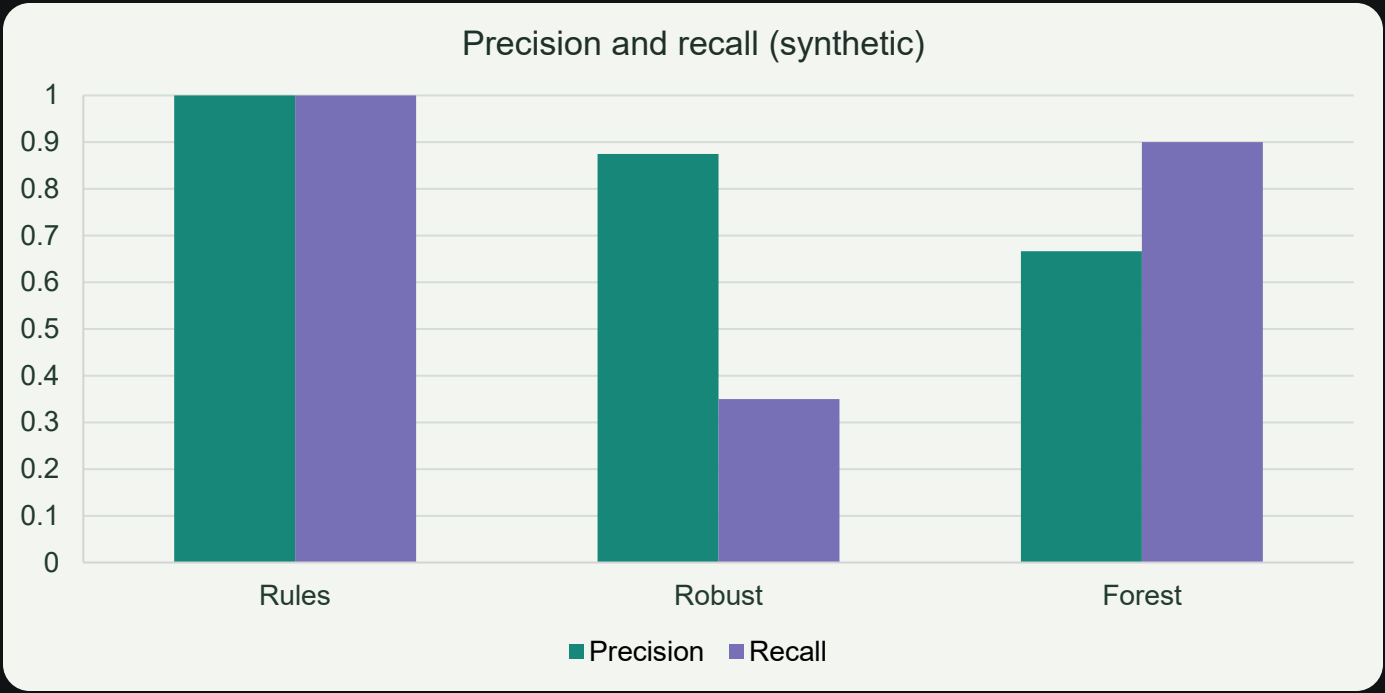
04
Rules / robust / forest

05
Score + explanation

Duplicates, irregular sampling and missing windows are handled in deterministic features.

Anomaly detection evaluation

Synthetic held-out windows only | n = 177; two fault incidents



Isolation Forest

Actual ↓ / Predicted →

	Normal	Anomaly
Normal	148	9
Anomaly	2	18

Rows: normal, anomaly. Small synthetic sample; no production accuracy claim.

Evidence-grounded incident correlation

Read-only analysis with an explicit uncertainty boundary

01 Bounded namespace/time grouping and deployment-change context

02 Historical similarity and four curated internal runbooks

03 12 expected directed links on eight synthetic fixtures; 8/8 retrieval matches

04 No remote LLM calls; no assistant path to privileged actions

Policy-driven recovery

A durable state machine, not a shell-command generator

01
Observe + qualify

02
Dry-run / reject

03
Signed approval

04
Apply bounded action

05
Verify / undo / escalate

Supported paths include rollout restart, bounded scale and rollback to a verified revision.

Safety and rollback architecture

Ambiguity stops automation

01 Protect critical namespaces; never delete arbitrary pods

02 Check target identity, policy version, approval expiry and action digest

03 Use leader election, durable intents and workload conflict fencing

04 Verify rollout and sampled service indicators; undo only within policy

05 Escalate stale incidents, exhausted budgets and unavailable APIs

Backend API architecture

Versioned contracts and durable operational state

01 /api/v1 health, readiness, version and typed operations endpoints

02 OIDC-compatible RBAC with namespace scope and audited mutations

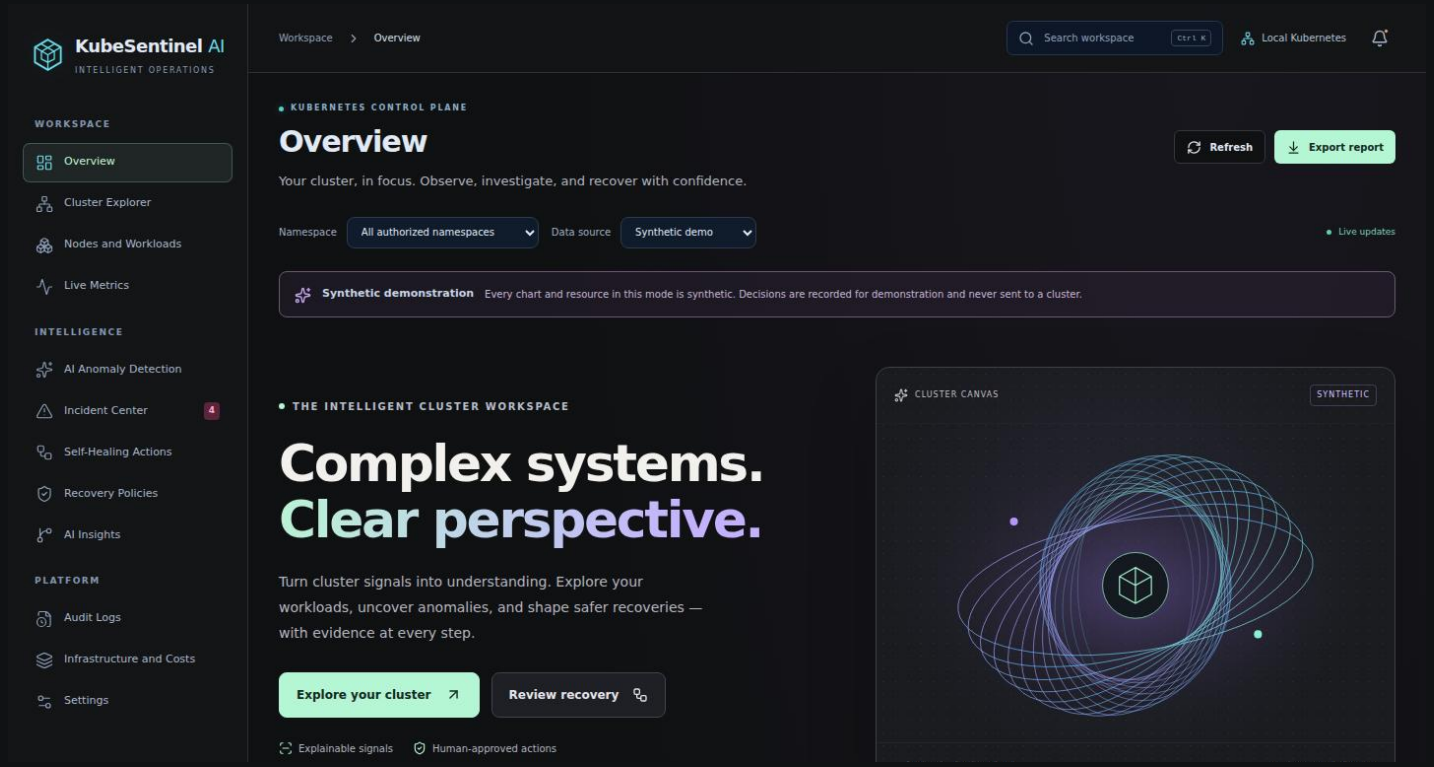
03 PostgreSQL migrations, bounded pools and durable queue records

04 Structured JSON logs, request correlation and trace context

05 SSE reconnect plus polling; explicit upstream outage responses

Operations dashboard

Actual browser capture | explicitly synthetic data partition



01 Twelve connected sections; responsive dark canvas

02 Inventory, metrics, incidents, policies and audit

03 Loading, offline, empty and error states

04 Real API mode remains the deployment default

Secure delivery and GitOps

Build and inspect before reviewed digest promotion

01
Lint + type + tests

02
Build + SBOM + scan

03
OIDC -> ECR digest

04
Reviewed environment PR

05
Argo reconciliation

Actions are SHA-pinned and tokens restricted.

Security and access control

Protect both the control plane and its evidence

01 Scoped user roles, exact origins and protected approval signing keys

02 Non-root containers, read-only filesystems and network policies

03 No committed credentials; OIDC replaces long-lived AWS keys

04 Telemetry redaction, bounded queries and fail-closed rate limits

05 Security scans remain release gates, including unfixed findings

Chaos testing methodology

Disposable environments and explicit destructive-test authorization

01 Isolate opted-in lab workloads; record context, namespace and source commit

02 Inject one fault; capture before/after state and action audit

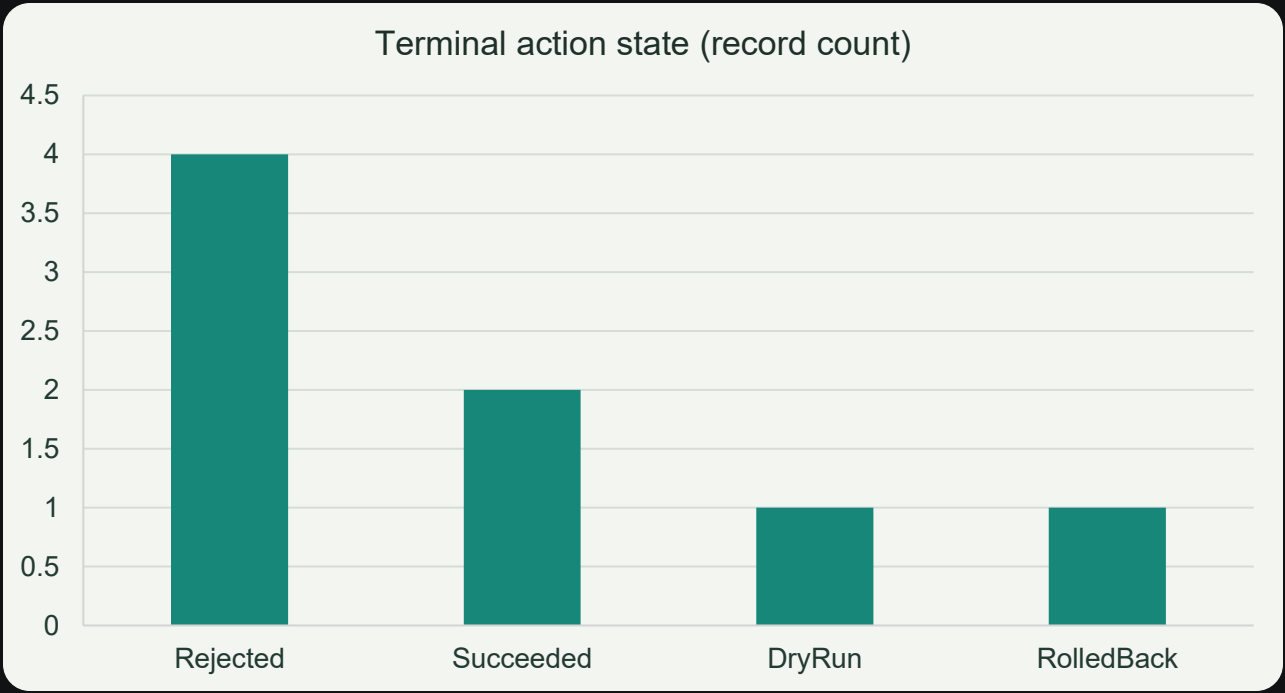
03 Assert dry-run, approval, cooldown, rejection and verified undo paths

04 Capture raw results; compare repeated runs against a native baseline

05 Node drain, network impairment and cloud chaos remain pending

Recovery evidence

One isolated kind run | eight action records



10 s

Observation → apply
Includes human approval

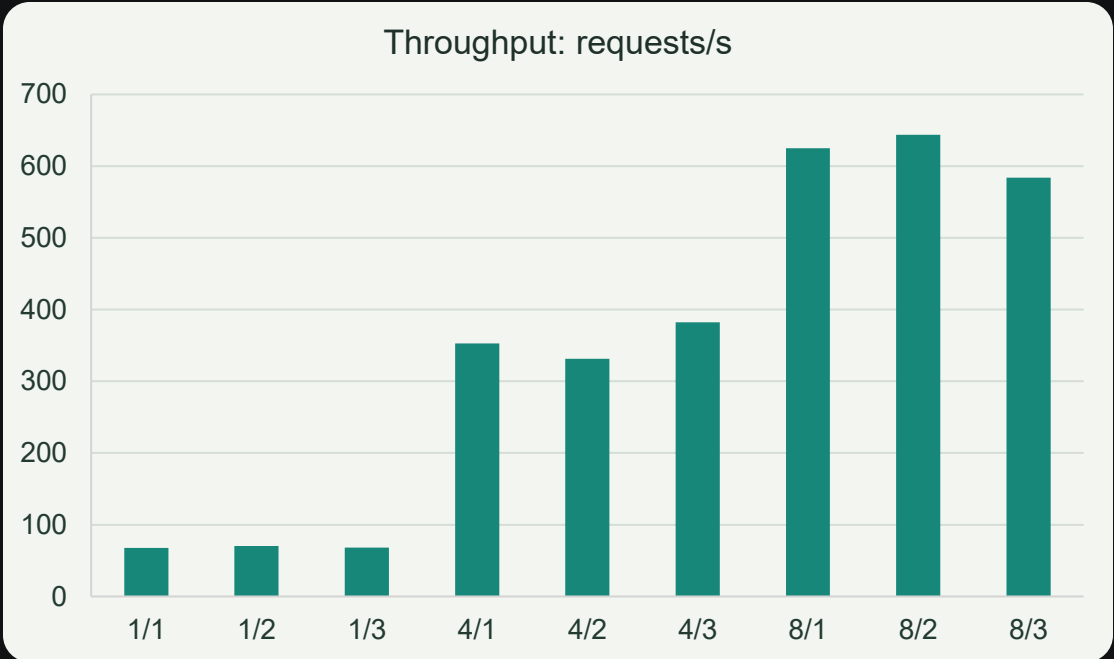
36 s

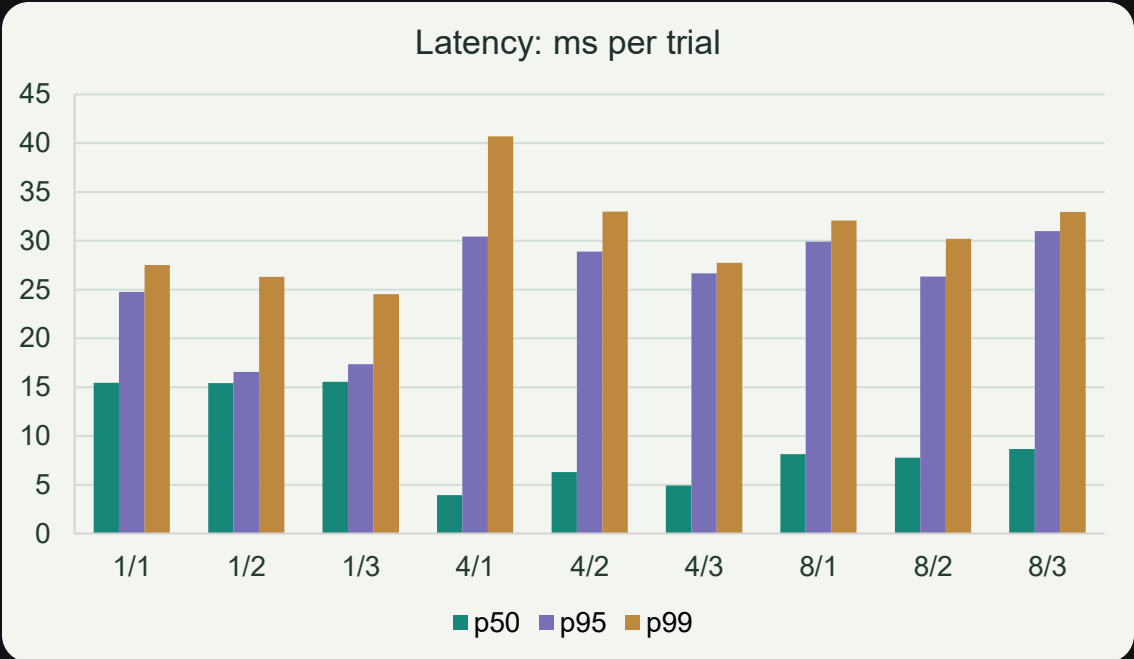
Verification interval
One approved scale example

One isolated run; no repeated baseline or recovery SLA.

Load, scalability and reliability

900 loopback health requests | Windows | no DB or cluster

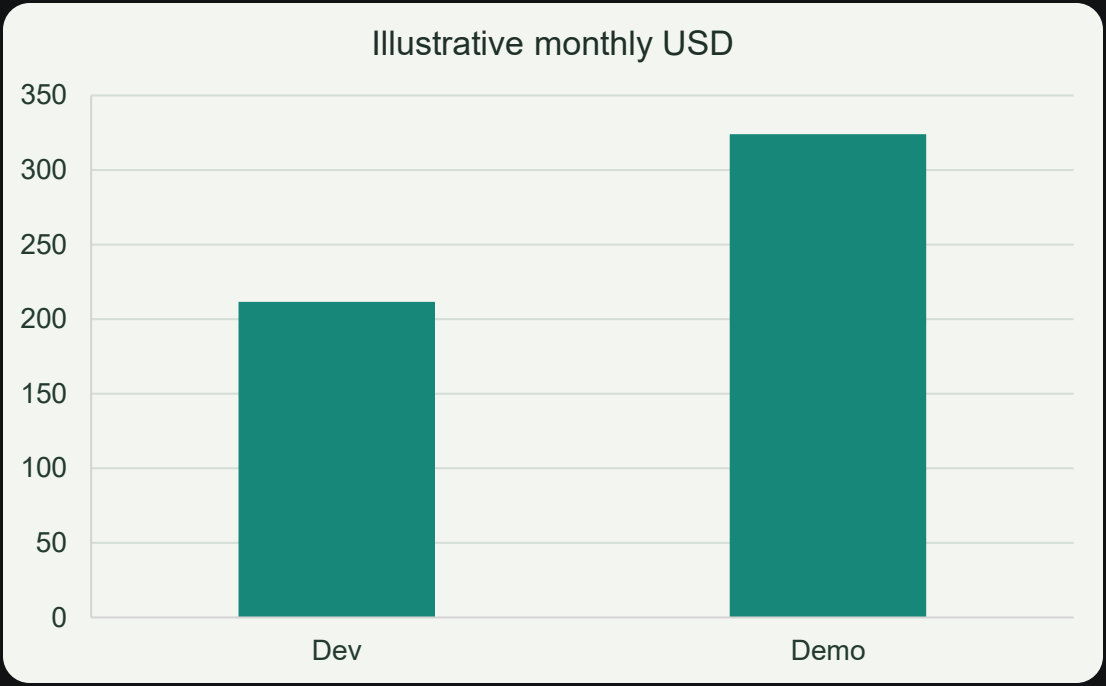




Labels = concurrency / repetition. Health only; no DB, cluster, CPU/memory or HPA measurement.

Cloud cost considerations

Illustrative planning inputs in USD; not an AWS quote



01 730 hours; EKS $0.10/h; nodes $0.10/h; NAT $0.05/h

02 Dev: $211.50/month; two-node demo: $324.00/month

03 Seven interface services across two AZs: +$102.20 before data

04 Excludes managed DB/cache, load balancers, transfer, tax and support

Budget inputs are assumptions, not current prices. Complete application costs are higher.

Known limitations and release gates

Portfolio evidence is narrower than production readiness

01 HIGH/CRITICAL image findings currently block v1.0.0

02 Synthetic ML and correlation fixtures need real labeled telemetry

03 Health-only load tests do not establish cluster capacity or SLOs

04 Live OIDC certification, cloud deployment and promotion are unverified

05 No measured CPU/memory overhead or repeated recovery comparison

Future roadmap

Close evidence gaps before expanding automation

01 First: remediate vulnerabilities and repeat all release checks

02 Next: repeated isolated fault trials and full-stack queue/load profiling

03 Then: live identity-provider validation and staging GitOps rehearsal

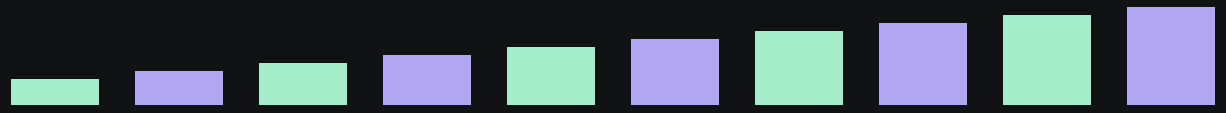
04 After approval: cost-bounded EKS demonstration and teardown proof

05 Later: real-label model evaluation, drift studies and read-only multi-cluster

Evidence, policy, verified recovery

A reproducible foundation for safer Kubernetes operations

Explore.
Reproduce.
Review.



Explore the source, runbooks and raw experiment records

Review the technical report and editable architecture sources

Contribute through a reviewed feature branch

github.com/manikantbindass/KubeSentinel-AI